

Bài giảng: Kiểu dữ liệu trong SQL & MySQL

Phân loại, Lựa chọn tối ưu, Miền giá trị & Thực hành thiết kế CSDL

Vũ Đức Minh

Trường Đại học Kinh tế Quốc dân (NEU), Hà Nội, Việt Nam

September 23, 2026

Mục tiêu bài giảng

Sau khi hoàn thành bài học này, người học có thể:

- Trình bày được khái niệm **kiểu dữ liệu (Data Type)** và vai trò trong RDBMS.
- Phân biệt chính xác các nhóm: **Numeric, String, Date/Time, Binary, Boolean & Special**.
- Nắm vững cơ chế lưu trữ của **CHAR vs VARCHAR, DECIMAL vs FLOAT, DATETIME vs TIMESTAMP**.
- Hiểu lý do bắt buộc dùng kiểu số thập phân chính xác (**DECIMAL**) cho tài chính/tiền tệ.
- Lựa chọn kiểu dữ liệu tối ưu dung lượng đĩa, bộ nhớ đệm RAM và chỉ mục (indexing).
- Vận dụng khai báo kiểu dữ liệu kết hợp ràng buộc trong câu lệnh **CREATE TABLE**.
- Nhận biết và khắc phục các lỗi thiết kế kiểu dữ liệu kinh điển trong thực tế.

Nội dung bài học

Phần 1: Khái niệm & Phân nhóm

- 1. Bản chất kiểu dữ liệu trong SQL
- 2. Sơ đồ các nhóm kiểu dữ liệu chính
- 3. Quy trình 5 bước lựa chọn kiểu
- 4. Kiểu số nguyên: Range & Unsigned
- 5. Số thực: DECIMAL vs FLOAT/DOUBLE

Phần 2: Ứng dụng & Tối ưu

- 6. Kiểu chuỗi: CHAR, VARCHAR, TEXT
- 7. Kiểu ngày giờ: DATETIME vs TIMESTAMP
- 8. Kiểu Binary, Boolean & JSON
- 9. Thực hành thiết kế & Sai lầm thường gặp
- 10. Câu hỏi ôn tập & Bài tập vận dụng

1. Bản chất kiểu dữ liệu trong SQL

Định nghĩa

Kiểu dữ liệu (Data Type) quy định **loại giá trị**, **miền giá trị (domain)**, **dung lượng bộ nhớ** và **các phép toán hợp lệ** trên một cột.

Vai trò cốt lõi trong RDBMS:

- **Toàn vẹn (Integrity)**: Chặn dữ liệu sai ngay từ tầng lưu trữ.
- **Tối ưu hiệu năng**: Giảm I/O đĩa, tăng dung lượng bản ghi trong Buffer Pool.
- **Tính toán chuẩn xác**: Đảm bảo phép so sánh, sắp xếp và tính toán không sai lệch.

Ba thuộc tính then chốt:

- **Miền giá trị**: Khoảng giá trị hợp lệ (vd: TINYINT $-128 \dots 127$).
- **Độ dài (Length)**: Số ký tự tối đa (VARCHAR(100)).
- **Độ chính xác (Precision/Scale)**: Số chữ số và phần thập phân (DECIMAL(10,2)).

2. Sơ đồ các nhóm kiểu dữ liệu phổ biến

SQL DATA TYPES	01	NUMERIC	INT, DECIMAL, FLOAT, NUMERIC
	02	BINARY	BINARY, BLOB
	03	STRING	CHAR, VARCHAR, TEXT
	04	BOOLEAN	BOOLEAN
	05	DATE/TIME	DATE, TIME, DATETIME
	06	SPECIAL	UUID, AON, XML, JSON

Hình 1: Cấu trúc phân nhóm kiểu dữ liệu phổ biến trong SQL

6 nhóm dữ liệu chính trong MySQL

- **Numeric:** Số nguyên (INT), số thực chính xác (DECIMAL), số gần đúng (FLOAT).
- **String:** Chuỗi ngắn (CHAR, VARCHAR), văn bản dài (TEXT).
- **Date/Time:** DATE, TIME, DATETIME, TIMESTAMP.
- **Binary:** BINARY, VARBINARY, BLOB.
- **Boolean:** BOOLEAN (bí danh cho TINYINT(1)).
- **Special:** JSON, GEOMETRY (Spatial GIS).

3. Quy trình 5 bước lựa chọn kiểu dữ liệu tối ưu

Bước 1: Xác định thực thể & thông tin cần lưu
(Họ tên, mã đơn hàng, ngày sinh, đơn giá, số dư)

Bước 2: Xác định bản chất ngữ nghĩa (Số đếm được, văn bản tìm kiếm, thời điểm, cờ logic)

Bước 3: Chọn nhóm kiểu phù hợp (Numeric, String, Temporal, Binary, JSON)

Bước 4: Xác định giới hạn & độ chính xác (INT UNSIGNED, VARCHAR(100), DECIMAL(10,2))

Bước 5: Bổ sung ràng buộc toàn vẹn (NOT NULL, CHECK, DEFAULT, UNIQUE)

4. Kiểu số nguyên (Integer Types) trong MySQL

Kiểu dữ liệu	Bộ nhớ	Miền giá trị (Signed)	Miền giá trị (Unsigned)	Ứng dụng tiêu biểu
TINYINT	1 byte	-128...127	0...255	Trạng thái, cờ boolean, số thứ tự nhỏ
SMALLINT	2 bytes	-32,768...32,767	0...65,535	Năm sản xuất, số phòng, số lượng nhỏ
MEDIUMINT	3 bytes	-8,388,608...8,388,607	0...16,777,215	Mã danh mục, bảng tra cứu vừa
INT	4 bytes	$\approx -2.14 \times 10^9 \dots 2.14 \times 10^9$	$0 \dots \approx 4.29 \times 10^9$	Khóa chính phổ biến, số lượng tồn kho
BIGINT	8 bytes	$\approx -9.22 \times 10^{18} \dots 9.22 \times 10^{18}$	$0 \dots \approx 1.84 \times 10^{19}$	Log giao dịch hệ thống lớn, Big Data

Nguyên tắc tối ưu hóa bộ nhớ

Tránh dùng BIGINT vô tội vạ cho mọi bảng! Một bảng 100 triệu dòng dùng BIGINT thay vì INT sẽ tiêu tốn thêm ≈ 400 MB chỉ cho riêng cột đó, làm chậm tốc độ quét index trên RAM.

5. Số thập phân chính xác vs Số thực gần đúng

DECIMAL (P, D) – Số chính xác

- Lưu trữ **chính xác tuyệt đối** từng chữ số dưới dạng nhị phân đóng gói.
- *P (Precision)*: Tổng chữ số (tối đa 65).
- *D (Scale)*: Số chữ số thập phân (tối đa 30).
- **Ví dụ**: DECIMAL (10, 2) → tối đa 99,999,999.99.
- **Bắt buộc cho**: Tiền tệ, lương, số dư tài khoản ngân hàng, hóa đơn.

FLOAT & DOUBLE – Số gần đúng

- Chuẩn IEEE 754 biểu diễn số thực.
- FLOAT: 4 bytes ($\approx$ 7 chữ số tin cậy).
- DOUBLE: 8 bytes ($\approx$ 15 chữ số tin cậy).
- Tính toán rất nhanh nhưng có **sai số làm tròn (rounding error)**.
- **Phù hợp cho**: Tọa độ GPS, cảm biến đo nhiệt độ, đo lường khoa học.

6. Kiểu chuỗi: CHAR vs VARCHAR

CHAR(n) – Chuỗi cố định

- Chiều dài luôn cố định là n ký tự ($0 \leq n \leq 255$).
- Ngắn hơn n sẽ được **đệm dấu cách** vào cuối.
- Truy xuất trực tiếp nhanh hơn vì offset cố định.
- **Ứng dụng**: Mã quốc gia ('VN', 'US'), mã tiền tệ ('VND'), mã bưu điện, hash MD5.

VARCHAR(n) – Chuỗi biến đổi

- Chiều dài linh hoạt theo nội dung thực tế (tối đa n ký tự).
- Tốn thêm **1 hoặc 2 bytes tiền tố** lưu độ dài chuỗi.
- Tiết kiệm dung lượng đĩa khi dữ liệu có độ dài chênh lệch lớn.
- **Ứng dụng**: Họ tên, email, địa chỉ, tiêu đề.

So sánh lưu trữ giá trị 'AB' (với bảng mã 1-byte)

7. Chuỗi văn bản dài (TEXT) & Bộ mã utf8mb4

Các kiểu văn bản lớn (TEXT)

- TINYTEXT: Tối đa 255 bytes.
- TEXT: Tối đa 64 KB (mô tả sản phẩm).
- MEDIUMTEXT: Tối đa 16 MB (nội dung bài báo).
- LONGTEXT: Tối đa 4 GB (tài liệu đồ sộ).
- Lưu trữ ngoài dòng (off-page) trong InnoDB khi nội dung lớn, tránh phân mảnh bảng.

Charset utf8mb4 & Collation

- **utf8mb4** là chuẩn UTF-8 đầy đủ của MySQL (1-4 bytes/ký tự).
- Hỗ trợ toàn diện tiếng Việt có dấu, chữ tượng hình và biểu tượng cảm xúc.
- utf8mb4_0900_ai_ci: Collation chuẩn từ MySQL 8.0, sắp xếp theo Unicode 9.0, không phân biệt hoa thường và không phân biệt dấu khi so sánh.

8. Kiểu ngày giờ: DATETIME vs TIMESTAMP

Kiểu	Bộ nhớ	Miền giá trị hợp lệ	Đặc tính Timezone & Ứng dụng
DATETIME	5 bytes	1000-01-01 9999-12-31	... Cố định: Không phụ thuộc múi giờ session. Phù hợp cho ngày sinh, hạn bảo hành, lịch hẹn.
TIMESTAMP	4 bytes	1970-01-01 2038-01-19 UTC	... Tự động đổi về UTC: Đổi theo múi giờ client khi đọc. Thích hợp cho audit log, created_at.
DATE	3 bytes	1000-01-01 9999-12-31	... Chỉ lưu năm-tháng-ngày (YYYY-MM-DD). Tiết kiệm cho ngày sinh, ngày hóa đơn.
TIME	3 bytes	-838:59:59 838:59:59	... Lưu giờ trong ngày hoặc khoảng thời gian trôi qua.

9. Kiểu Binary, Boolean & JSON hiện đại

BLOB vs VARBINARY

- BINARY / VARBINARY: Chuỗi byte thô cố định hoặc linh hoạt (lưu hash mật khẩu, UUID binary 16 bytes).
- BLOB: Lưu file nhị phân (ảnh, file đính kèm).
- *Thực tế tốt nhất*: Nên lưu file media trên Cloud Storage (S3, CDN) và chỉ lưu URL chuỗi trong database.

BOOLEAN & JSON

- Trong MySQL, BOOLEAN là alias của TINYINT(1) (0 = FALSE, 1 = TRUE).
- JSON: Kiểu dữ liệu gốc (MySQL 5.7+). Tự động validate cú pháp JSON, hỗ trợ index trên virtual column và truy vấn nhanh qua toán tử -> và ->>.

10. Thực hành: Khai báo đa dạng kiểu dữ liệu trong bảng

```
CREATE TABLE products (
    product_id    INT UNSIGNED AUTO_INCREMENT PRIMARY KEY, -- Khoa chính số nguyên dương
    product_code  CHAR(8) NOT NULL UNIQUE,                -- Mã sản phẩm có định 8 ký tự
    product_name  VARCHAR(120) NOT NULL,                  -- Tên sản phẩm linh hoạt
    category_id   SMALLINT UNSIGNED NOT NULL,             -- Mã danh mục < 65,535
    cost_price    DECIMAL(10,2) NOT NULL,                  -- Giá vốn: chính xác tuyệt đối
    sale_price    DECIMAL(10,2) NOT NULL,                  -- Giá bán: chính xác tuyệt đối
    weight_kg     FLOAT(5,2),                              -- Cân nặng: chấp nhận sai số nhỏ
    description   TEXT,                                    -- Mô tả bài viết dài
    attributes    JSON,                                    -- Thông số kỹ thuật dòng
    is_active     BOOLEAN NOT NULL DEFAULT TRUE,          -- Còn kinh doanh (TINYINT(1))
    created_at    DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP -- Mốc thời gian tạo
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
```

11. Các sai lầm kinh điển khi chọn kiểu dữ liệu

Sai lầm thiết kế phổ biến

- **Lưu số điện thoại bằng INT:** Bị mất số 0 đầu (0912... thành 912...), mất dấu +. Luôn dùng VARCHAR(20).
- **Lưu tiền bằng FLOAT:** Dễ gặp sai số làm tròn số học ($0.1 + 0.2 \neq 0.3$). Luôn dùng DECIMAL.
- **Lưu ngày tháng bằng VARCHAR:** Không so sánh ngày chuẩn, không tính toán khoảng cách ngày. Dùng DATE.

Lạm dụng và lãng phí

- **Lạm dụng VARCHAR(255) mọi nơi:** Tốn bộ nhớ RAM nội bộ khi sắp xếp ORDER BY hoặc tạo bảng tạm.
- **Dùng BIGINT cho cờ nhị phân:** Lãng phí bộ nhớ gấp 8 lần so với TINYINT hoặc BOOLEAN.
- **Lưu video/file to vào BLOB:** Gây phình to kích thước database, chậm backup và restore hệ thống.

12. Câu hỏi kiểm tra nhanh (Quick Quiz)

Câu 1: Chọn kiểu cho đơn giá

Cột lưu đơn giá bán hàng nên khai báo kiểu nào?

- A. FLOAT
- B. **DECIMAL(10,2)** (Chính xác!)
- C. VARCHAR(50)
- D. BIGINT

Câu 2: Số điện thoại khách hàng

Vì sao nên dùng VARCHAR cho số điện thoại?

- A. Số điện thoại không dùng tính toán.
- B. Giữ nguyên số 0 đầu và dấu +.
- C. Tránh nguy cơ tràn số nguyên.
- D. **Cả A, B và C đều đúng.**

Câu 3: Phân biệt CHAR và VARCHAR

Hỏi: Khi lưu mã quốc gia 2 ký tự ('VN', 'JP', 'US'), nên chọn CHAR(2) hay VARCHAR(2)?

13. Bài tập vận dụng thiết kế bảng

Bài tập thực hành tại lớp

Hãy viết câu lệnh CREATE TABLE cho bảng student_profiles để quản lý hồ sơ sinh viên với các yêu cầu:

1. **Mã sinh viên:** Số nguyên dương tự tăng, làm khóa chính.
2. **Mã định danh SV:** Chuỗi đúng 10 ký tự (CHAR(10)), không trùng lặp.
3. **Họ và tên:** Tối đa 100 ký tự có dấu tiếng Việt (VARCHAR(100)).
4. **Ngày sinh:** Chỉ cần lưu ngày tháng năm (DATE).
5. **Điểm trung bình (GPA):** Thang điểm 4 hoặc 10, có 2 chữ số thập phân (DECIMAL(4,2)).
6. **Tình trạng nợ học phí:** Giá trị logic Đúng / Sai (BOOLEAN).
7. **Thời điểm nộp hồ sơ:** Tự động ghi nhận mốc thời gian hệ thống (DATETIME).

Tổng kết bài học

Ba nguyên tắc vàng ghi nhớ

- **Đúng bản chất:** Số dùng để tính, chuỗi dùng để đọc, thời gian dùng để đo lường.
- **Vừa đủ phạm vi:** Tránh chọn quá nhỏ gây tràn dữ liệu, tránh chọn quá to gây lãng phí RAM.
- **Chính xác tuyệt đối:** Dữ liệu tài chính, tiền tệ bắt buộc dùng DECIMAL.

Tài liệu học tập tiếp theo

- Bài tiếp theo: **CREATE TABLE & Ràng buộc toàn vẹn** (Primary Key, Foreign Key, Check, Unique).
- Lab thực hành: Kiểm tra dung lượng trên đĩa của các kiểu dữ liệu qua `information_schema`.